
Test Summary & Quality Evaluation Report

IBA Campus Facility Booking System (CFBS) — Cycle 1 Summary

Prepared by:	Muhammad Imad Raza (26953) Abdul Wasay Imran (27126) Saad Imam (27079) Syed Muhammad Deebaj Haider Kazmi (26012)
Reviewed & Approved by:	Muhammad Imad Raza (26953)
Target System:	NestJS REST Backend / React + Vite Frontend
Testing Environment:	Sandbox Sandbox-Supabase (PostgreSQL)
Document Version:	1.0 (Final Evaluation)
Date:	May 28, 2026

Executive Summary & Release Recommendation

This report provides a formal assessment of the quality state of the IBA Campus Facility Booking System (CFBS) following the completion of Test Cycle 1. Testing was performed across three execution boundaries: isolated unit testing, integration-level REST API testing, and end-to-end (E2E) automated browser verification.

Based on the quantitative quality criteria defined in the CFBS Master Test Plan, the system is **not approved for release** to production. High-priority functional gaps, including four Severity-1 (Critical) database and validation defects, remain unresolved. Release sign-off is withheld pending a full regression cycle of the remediated codebase.

1 Scope & Methodology

The primary objective of this testing cycle was to validate the functional integrity, security access boundaries, database transaction safety, and user-experience paths of the CFBS against the baseline Software Requirements Specification (SRS).

The target of evaluation (TOE) comprised:

- **CFBS REST API Backend:** NestJS micro-architecture utilizing a remote, isolated PostgreSQL container (Supabase).
- **CFBS Client Application:** Single-page React application compiled via Vite.

Testing was performed programmatically on an isolated loopback configuration. Database states were managed via transaction truncations and hardcoded static UUID injection (`seedTestData.sql`) before the execution of each isolated test suite to guarantee strict repeatability.

2 Requirements Coverage Analysis

A Comprehensive Traceability Map was maintained during this cycle to track the verification of the 37 distinct functional requirements defined in the CFBS SRS.

- **Requirements Mapped to Tests:** 100% (37 out of 37 requirements were associated with at least one automated unit, integration, or E2E test case).
- **Requirements Successfully Verified:** 75.6% (28 out of 37 requirements passed all associated assertions).
- **Unverified/Failed Requirements:** 24.4% (9 requirements failed to resolve successfully due to active code-level defects).

The non-functional security requirements (Role-Based Access Control) were verified successfully. However, critical transaction-safety, boundary-limit, and state-machine requirements failed, as detailed in Section 4.

3 Test Execution Statistics

The CFBS test suite comprised 124 distinct test points distributed across three levels of the execution harness.

3.1 Execution Aggregates

The following table provides the quantitative pass/fail breakdown of the test runs:

3.2 Module-Wise Performance

The test runs were distributed across six functional modules to evaluate quality density and isolate weak architectural layers:

Table 1: Test Run Performance Summary

Test Level	Harness	Executed	Passed	Failed	Pass Rate
Unit Level	NestJS / Jest	39	39	0	100.0%
Integration Level	Supertest / Express	43	34	9	79.07%
End-to-End (E2E)	Playwright (Chrome/FF)	42	35	7	83.33%
Total Aggregates		124	108	16	87.09%

Table 2: Module Performance Profile

Functional Module	Executed Tests	Passed	Failed	Pass Rate
Authentication & Authorization (TC-AUTH)	28	28	0	100.0%
Room Booking Creation (TC-BOOK)	16	12	4	75.00%
PO Dashboard Management (TC-PO)	13	11	2	84.62%
Booking Cancellation (TC-CANCEL)	22	13	9	59.09%
Admin Infrastructure Management (TC-ADMIN)	44	44	0	100.0%
Data Integrity & Audit Trails (TC-DATA)	1	0	1	0.00%

4 Technical Defect Analysis

During this execution cycle, a total of 15 unique defects were registered. The team classified these defects according to standard severity guidelines to understand their impact on core business flows.

4.1 Defect Severity and Density Distribution

- **Severity-1 (Critical):** 4 defects (Functional blockers, database transaction crashes, or core constraint failures).
- **Severity-2 (Major):** 9 defects (Major workflow deviations, missing cancellation buttons, or audit trail omissions).
- **Severity-3 (Minor):** 2 defects (Configuration out-of-bounds or missing non-critical schema roles).
- **Severity-4 (Trivial):** 0 defects (Cosmetic or minor spelling anomalies).

4.2 Analysis of Core Architectural Failures

The QA team isolated three main root causes that contributed to the majority of integration and end-to-end test failures.

4.2.1 1. Database Constraint Inflexibility (DEF-002)

The integration concurrency test (TC-BOOK-009) and the E2E PO conflict resolution test (TC-PO-004) failed due to an unconditional unique constraint in `supabase-schema.sql`:

```
UNIQUE (room_id, date, slot_id)
```

Table 3: Defect Matrix Summary

Defect ID	Defect Summary Title	Severity	Status
DEF-001	Cancelled bookings are incorrectly marked as 'rejected'	Severity-2	Open
DEF-002	Unconditional unique DB constraint blocks slot recycling	Severity-1	Open
DEF-003	Missing validation allows room bookings for past dates	Severity-2	Open
DEF-004	Approved bookings on student dashboard lack Cancel option	Severity-2	Open
DEF-005	Missing current-year constraint checks allow future reservations	Severity-2	Open
DEF-006	Time slot definitions violate SRS-defined business hours	Severity-3	Open
DEF-007	Lack of temporal check allows cancellation of elapsed bookings	Severity-2	Open
DEF-008	Disable Room requirement implemented only as slot blocks	Severity-2	Open
DEF-009	Missing 'Faculty/Teacher' user role in system schemas	Severity-3	Open
DEF-010	Missing range validation on slot ID crashes the database	Severity-1	Open
DEF-011	Cascade deletes destroy historical audit trails	Severity-1	Open
DEF-012	Student cancellation overwrites reviewer ID values	Severity-2	Open
DEF-013	Missing pagination on database query returns entire history	Severity-2	Open
DEF-014	Room Booking purpose input field vulnerable to stored XSS	Severity-2	Open
DEF-015	API SELECT projection explicitly omits reviewed.by field	Severity-1	Open

This constraint makes no distinction between active (pending, approved) and inactive (rejected, cancelled) bookings. As a result, once a slot is booked, any secondary pending request for the same slot is rejected immediately with a database error, rather than allowing multiple pending requests to reach the PO for review. Furthermore, if a booking is cancelled, that slot can never be re-booked, which prevents slot recycling.

4.2.2 2. Missing Range Validation on Inputs (DEF-010)

The boundary integration tests (TC-B00K-004 and TC-B00K-007) failed because out-of-bounds slot IDs (such as `slot_id: 0` or `slot_id: 8`) bypassed the NestJS validation layer. Because the `CreateBookingDto` only enforces a basic `@IsInt()` decorator, invalid values hit the database directly, resulting in an unhandled `500 Internal Server Error` instead of a clean `400 Bad Request` validation response.

4.2.3 3. Explicit Audit Trail Omission (DEF-015)

During data integrity verification (TC-DATA-001), the test suite attempted to verify the `reviewed_by` column in a retrieved booking. This assertion failed because the database projection string in `bookings.service.ts` explicitly excludes the `reviewed_by` field:

```
const SELECT = `id, date, slot_id, purpose, status, created_at, updated_at,
  users!bookings_user_id_fkey(id, erp, name, email),
  rooms(id, name, buildings(id, name)),
  time_slots(id, start_time, end_time, label)`;
```

By omitting this field from SELECT operations, the backend prevents the UI from displaying the required audit history showing which staff member approved or rejected each request.

5 Deviations from the Test Plan

Three notable deviations from the Master Test Plan occurred during Cycle 1 execution:

1. **Use of Playwright instead of Selenium:** Playwright was chosen over selenium due to its ease in learning and better features
2. **E2E Browser Cleanup Warnings (Firefox):** Playwright tests executed on the Firefox engine occasionally threw context termination warnings (*"Browser.removeBrowserContext: SSI is undefined"*). This was isolated to a browser-cleanup protocol error rather than CFBS client logic, so these warnings did not affect the functional pass-rate assessment.
3. **Temporal Boundaries (Manual Verification):** Automated checks for past-date cancellations (TC-CANCEL-004) were supplemented with manual API requests, as managing real-time system clock values in the testing database is highly prone to timing issues.

6 Exit Criteria Assessment & Release Recommendation

To determine release readiness, the Cycle 1 execution metrics were evaluated systematically against the exit criteria defined in Section 9.2 of the Master Test Plan.

Table 4: Exit Criteria Compliance Evaluation

Metric Name	Target Threshold	Actual Value	Compliance Status
Unit Pass Rate	100.0%	100.0% (39/39)	COMPLIANT
Integration Pass Rate	≥ 95.0%	79.07% (34/43)	NON-COMPLIANT
E2E Pass Rate	≥ 90.0%	83.33% (35/42)	NON-COMPLIANT
Severity-1 Resolution	100.0% Resolved	4 Active Defects	NON-COMPLIANT
Severity-2 Resolution	≥ 85.0% Resolved	0% Resolved	NON-COMPLIANT
Requirements Traceability	100.0% Mapped	100.0% Mapped	COMPLIANT

Deficiency Summary: The quality state of the application does not meet the standards required for release. Four critical Severity-1 defects remain open:

- Unconditional unique constraints that prevent slot recycling (DEF-002).
- Missing range validation on inputs causing database crashes (DEF-010).
- Cascade deletions that erase historical audit trails (DEF-011).
- Omissions of crucial audit fields from API SELECT projections (DEF-015).

Additionally, the overall test pass rate of 87.09% falls below the combined minimum threshold of 95.0% required to assure transaction and state-machine stability.

Final Release Recommendation: NO-GO.

It is recommended that development resources be focused on fixing the unique index constraints, validation decorators, and cascading foreign keys. Once these code changes are complete, a full regression cycle must be executed to verify the fixes and confirm that the system meets all release criteria.